Informe de pruebas

Proyecto: Clasificador de congestión de estaciones de TransMilenio (Bogotá)
Curso: Inteligencia Artificial, Universidad Iberoamericana
Actividad: 3 - Métodos de aprendizaje supervisado

1. Objetivo

Verificar que el generador de datos, el entrenamiento y el predictor se comportan como se documenta, y reportar con evidencia real las métricas del modelo. Todas las salidas de este informe se obtuvieron ejecutando el código del repositorio con semilla 42.

2. Entorno

Elemento	Valor
Sistema operativo	macOS (Darwin)
Python	3.14.7
pytest	9.1.1
Bibliotecas	pandas, scikit-learn, matplotlib, pytest (`requirements.txt`)

3. Resultados de `python -m pytest -v`

Total: 19 pruebas, 19 aprobadas, 0 fallidas (6.63 s).

```
$ .venv/bin/python -m pytest -v
tests/test_dataset.py::test_schema_and_size PASSED                       [  5%]
tests/test_dataset.py::test_same_seed_is_deterministic PASSED            [ 10%]
tests/test_dataset.py::test_different_seed_differs PASSED                [ 15%]
tests/test_dataset.py::test_all_classes_present_in_reasonable_proportion PASSED [ 21%]
tests/test_dataset.py::test_is_transfer_matches_network PASSED           [ 26%]
tests/test_predict.py::test_missing_model_is_trained_on_the_fly PASSED   [ 31%]
tests/test_predict.py::test_resolve_station_ignores_case_and_accents PASSED [ 36%]
tests/test_predict.py::test_resolve_station_unknown_suggests_close_names PASSED [ 42%]
tests/test_predict.py::test_weekday_rush_hour_transfer_station_in_rain_is_high PASSED [ 47%]
tests/test_predict.py::test_sunday_night_non_transfer_station_is_low PASSED [ 52%]
tests/test_predict.py::test_output_lists_probabilities_for_every_level PASSED [ 57%]
tests/test_predict.py::test_unknown_station_exits_2_with_suggestion PASSED [ 63%]
tests/test_predict.py::test_hour_out_of_range_is_rejected PASSED         [ 68%]
tests/test_train.py::test_load_data_returns_features_and_target PASSED   [ 73%]
tests/test_train.py::test_split_is_stratified_80_20 PASSED               [ 78%]
tests/test_train.py::test_tree_beats_baseline_by_clear_margin PASSED     [ 84%]
tests/test_train.py::test_training_is_deterministic PASSED               [ 89%]
tests/test_train.py::test_best_params_are_searched_values PASSED         [ 94%]
tests/test_train.py::test_main_saves_model_and_plot PASSED               [100%]

============================== 19 passed in 6.63s ==============================
```

Qué verifica cada prueba

Módulo	Prueba	Qué verifica
`test_dataset.py`	`test_schema_and_size`	3000 filas, columnas exactas, horas entre 5 y 22, valores válidos de `day_type`, `weather` y `near_event`, sin nulos
	`test_same_seed_is_deterministic`	La misma semilla produce un DataFrame idéntico
	`test_different_seed_differs`	Una semilla distinta produce datos distintos

Módulo	Prueba	Qué verifica
	`test_all_classes_present_in_reasonable_proportion`	Las tres clases aparecen y ninguna baja de 15 %
	`test_is_transfer_matches_network`	Cada línea de una fila sirve a su estación e `is_transfer` coincide con la red; ambos valores aparecen
`test_train.py`	`test_load_data_returns_features_and_target`	`load_data` devuelve las columnas de `FEATURES` y la etiqueta con igual longitud
	`test_split_is_stratified_80_20`	Partición 80/20 con proporciones de clase iguales (tolerancia 0.01) en entrenamiento y prueba
	`test_tree_beats_baseline_by_clear_margin`	El árbol supera a la línea base en al menos 0.25 de exactitud; `evaluate` devuelve exactitud, reporte, matriz y etiquetas
	`test_training_is_deterministic`	Dos entrenamientos dan los mismos hiperparámetros y las mismas predicciones
	`test_best_params_are_searched_values`	Los hiperparámetros elegidos pertenecen a la rejilla de búsqueda
	`test_main_saves_model_and_plot`	`main` imprime las métricas, guarda la gráfica y un modelo que se puede cargar y usar
`test_predict.py`	`test_missing_model_is_trained_on_the_fly`	Sin modelo guardado, `load_model` entrena y lo guarda
	`test_resolve_station_ignores_case_and_accents`	`heroes` -> `Héroes`, `NORMANDIA` -> `Normandía`
	`test_resolve_station_unknown_suggests_close_names`	Estación desconocida lanza error con sugerencias
	`test_weekday_rush_hour_transfer_station_in_rain_is_high`	Ricaurte, día hábil, 7 h, lluvia -> `alta`
	`test_sunday_night_non_transfer_station_is_low`	Calle 100, domingo, 21 h -> `baja`
	`test_output_lists_probabilities_for_every_level`	La salida lista alta, media y baja
	`test_unknown_station_exits_2_with_suggestion`	Código de salida 2, nada en stdout, mensaje con sugerencia en stderr
	`test_hour_out_of_range_is_rejected`	`--hour 3` termina con código 2

Las pruebas de `test_predict.py` redirigen `MODEL_PATH` a un directorio temporal, por lo que no tocan `models/`.

4. Resultados de `python -m src.train`

Salida real (se omiten las líneas del árbol completo; el resto es textual):

```
$ .venv/bin/python -m src.train
Rows: 3000 | train: 2400 | test: 600
Chosen hyperparameters: {'max_depth': 6, 'min_samples_leaf': 10}
Baseline accuracy (most frequent class): 0.410
Tree accuracy (held-out test): 0.787
5-fold CV accuracy (train split): 0.759 (+/- 0.019)

Classification report:
              precision    recall  f1-score   support

        high       0.81      0.89      0.85       167
         low       0.84      0.86      0.85       246
```

```
      medium       0.68      0.60      0.64       187

    accuracy                           0.79       600
   macro avg       0.78      0.78      0.78       600
weighted avg       0.78      0.79      0.78       600

Confusion matrix (rows = true, columns = predicted, order = ['high', 'low', 'medium']):
[[148   0  19]
 [  0 212  34]
 [ 35  40 112]]

Top feature importances:
  hour                    0.430
  day_type_weekday        0.235
  is_transfer             0.125
  day_type_saturday       0.075
  near_event              0.065
  weather_dry             0.046

Decision rules:
|--- day_type_weekday <= 0.50
|   |--- day_type_saturday <= 0.50
|   |   |--- is_transfer <= 0.50
...
Model saved to .../models/congestion_tree.joblib
Tree plot saved to .../docs/img/arbol-decision.png
```

La salida completa tiene 167 líneas; las reglas completas se obtienen ejecutando el comando. La gráfica de los primeros niveles está en `docs/img/arbol-decision.png`.

5. Pruebas funcionales del predictor

Ejecutadas desde la raíz del repositorio.

Caso	Comando	Salida real	Código
P1 hora pico, estación de entrada, lluvia	`--station "Calle 100" --hour 7 --day weekday --weather rain --event 0`	alta 0.79, media 0.21, baja 0.00 -> `alta`	0
P2 hora pico, transbordo, lluvia, sin tildes ni mayúsculas	`--station ricaurte --hour 7 --day weekday --weather rain`	alta 1.00 -> `alta`	0
P3 domingo en la noche	`--station "Calle 100" --hour 21 --day sunday_holiday`	baja 1.00 -> `baja`	0
P4 sábado al mediodía	`--station Héroes --hour 12 --day saturday`	alta 0.00, media 0.26, baja 0.73 -> `baja`	0
P5 estación desconocida	`--station "Calle 2" --hour 7`	error con sugerencias	2
P6 hora fuera de rango	`--station "Calle 100" --hour 3`	error de `argparse`	2

Salidas textuales:

```
$ python -m src.predict --station "Calle 100" --hour 7 --day weekday --weather rain --event 0
Estación: Calle 100 | Hora: 7 | Día: weekday | Clima: rain | Evento: 0
Congestión prevista: alta
  alta: 0.79
  media: 0.21
  baja: 0.00

$ python -m src.predict --station "Calle 100" --hour 21 --day sunday_holiday
Estación: Calle 100 | Hora: 21 | Día: sunday_holiday | Clima: dry | Evento: 0
Congestión prevista: baja
  alta: 0.00
  media: 0.00
  baja: 1.00

$ python -m src.predict --station Héroes --hour 12 --day saturday
Estación: Héroes | Hora: 12 | Día: saturday | Clima: dry | Evento: 0
```

```
Congestión prevista: baja
  alta: 0.00
  media: 0.26
  baja: 0.73

$ python -m src.predict --station "Calle 2" --hour 7
Estación desconocida: 'Calle 2'. ¿Quisiste decir: Calle 72, Calle 26, Calle 75?
(código de salida 2)

$ python -m src.predict --station "Calle 100" --hour 3
usage: python -m src.predict [-h] --station STATION --hour 5-22
                             [--day {weekday,saturday,sunday_holiday}]
                             [--weather {dry,rain}] [--event {0,1}]
python -m src.predict: error: argument --hour: invalid choice: '3' (choose from '5', '6', ... '22')
(código de salida 2)
```

Las probabilidades de la salida se promedian entre las líneas que sirven a la estación y se redondean a dos decimales, por lo que P4 suma 0.99.

6. Interpretación de las métricas

Métrica	Valor	Lectura
Línea base (clase más frecuente)	0.410	Lo que se obtiene sin usar ninguna variable; el árbol debe superarlo
Árbol, partición de prueba	0.787	+0.377 sobre la línea base
Validación cruzada, 5 particiones	0.759 (+/- 0.019)	Estimación más estable que una sola partición; la desviación es baja
F1 `high` / `low` / `medium`	0.85 / 0.85 / 0.64	`medium` es la clase más débil

Línea base frente al árbol. El árbol mejora 37.7 puntos porcentuales sobre predecir siempre `low`, lo que indica que aprende estructura real del conjunto y no solo la proporción de clases.

Validación cruzada. El promedio de validación cruzada (0.759) es algo menor que la exactitud en la partición de prueba (0.787). La diferencia, 0.028, es cercana a 1.5 desviaciones estándar de la validación cruzada; es razonable atribuirla a la variabilidad de una sola muestra de 600 filas y no a sobreajuste. La desviación de 0.019 muestra que el resultado es estable entre particiones.

Matriz de confusión. Filas = clase real, columnas = predicha (orden `high`, `low`, `medium`):

Real \ Predicha	`high`	`low`	`medium`
`high`	148	0	19
`low`	0	212	34
`medium`	35	40	112

- Los errores ocurren solo entre clases vecinas: nunca se confunde `high` con `low` (ambas celdas valen 0).
- `medium` concentra los errores: 75 de sus 187 filas reales se clasifican mal (recall 0.60), y recibe 53 falsos positivos de las otras dos clases (precisión 0.68).

Por qué `medium` es la clase más débil. Es una banda intermedia entre dos umbrales (0.5 y 1.8) del puntaje latente, con ruido gaussiano de desviación 0.5. Una fila cuyo puntaje sin ruido queda cerca de un umbral cruza al otro lado por el ruido, y esas filas son frecuentes en `medium` porque tiene dos fronteras, mientras que `low` y `high` tienen una. Ninguna variable observada contiene ese ruido, así que parte de ese error no es reducible por el modelo.

Variables importantes. `hour` (0.430), `day_type` (0.235 para día hábil y 0.075 para sábado), `is_transfer` (0.125), `near_event` (0.065) y `weather` (0.046). Coincide con el generador, que da más peso a la hora y el tipo de día. Las reglas impresas también incluyen divisiones sobre `station_*` y `line_*` (por ejemplo

`station_Calle 26`, `line_Caracas`), que el generador no usa directamente: son divisiones espurias por la codificación one-hot.

7. Advertencia sobre la validez de la exactitud

La exactitud de 0.787 mide qué tan bien el árbol **recupera las reglas sintéticas** escritas en `src/dataset.py` (más ruido). No mide congestión real de TransMilenio, porque los datos no provienen de mediciones. Estos resultados demuestran que el pipeline (generación, partición, selección por validación cruzada, evaluación y predicción) funciona y que el árbol es interpretable; no permiten afirmar que el modelo pronostique la congestión de la red real. Ver `docs/descripcion-datos.md` para el plan de sustitución por datos reales.

8. Resumen

Aspecto	Resultado
Pruebas automatizadas	19 de 19 aprobadas
Mejora sobre la línea base	0.787 frente a 0.410
Validación cruzada	0.759 (+/- 0.019)
Clase más débil	`medium` (F1 0.64)
Errores de predicción de alta a baja o viceversa	0 en la partición de prueba
Manejo de errores del CLI	Estación desconocida y hora fuera de rango terminan con código 2